

Paper 05 — Protocollo di Comunicazione

WebSocket, OSC e il Bridge C++↔React

WHYCREMISI RESEARCH PAPERS – N.05
Protocollo di Comunicazione Interno

"Il linguaggio che fa parlare il codice con la musica."

Categoria: Protocollo Tecnico

Versione protocollo: JSON v1.1

1. Panoramica

WhyCremisi usa **due protocolli** in parallelo per coprire tutti gli scenari:

WebSocket (JSON)	↔	React UI ↔ C++ Bridge
OSC (UDP/TCP)	↔	C++ Bridge ↔ DAW Host / Plugin Esterni

Il JSON WebSocket è il protocollo principale. OSC è usato per la comunicazione con DAW che lo supportano nativamente (Reaper, TouchOSC).

2. Protocollo WebSocket — Struttura Messaggi

Ogni messaggio è un frame JSON text:

```
{
  "type":      "categoria.azione",
  "id":        "uuid-v4-opzionale",
  "timestamp": 1716500000000,
  "payload":   { ... }
}
```

3. Dizionario Completo Messaggi

→ UI verso Plugin (Client → Server)

```
ai.prompt
```

```
payload: {
  "prompt": "string",
  "provider": "gemini|ollama|openai|anthropic"
}
```

Descrizione: Invia un prompt testuale all'AI

daw.command

```
payload: {
  "command": "play|stop|record|pause|setTempo|
             setVolume|setPan|muteTrack|soloTrack|
             setGain|setDrive"
  "bpm": 128.0,          (solo setTempo)
  "trackId": 1,          (per comandi traccia)
  "valueDb": -6.0,       (per setVolume/setGain)
  "value": 0.75,         (per setDrive/setPan)
  "muted": true          (per muteTrack)
}
```

plugin.control ← NUOVO (v1.1)

```
payload: {
  "trackId": 1,
  "pluginSlot": 0,
  "pluginName": "FabFilter Pro-Q3",
  "paramName": "Band1_Gain",
  "value": -4.5,
  "unit": "dB"
}
```

Descrizione: Modifica parametro su plugin di terze parti

plugin.query ← NUOVO (v1.1)

```
payload: {
  "trackId": 1,
  "pluginSlot": 0,
  "paramName": "Band1_Gain"    (opzionale, ometti per tutto)
}
```

Risposta: plugin.state con valori attuali

config.set

```
payload: {
  "key": "ai.provider|ai.apiKey|ai.model|
         ui.theme|ui.language",
  "value": "gemini",
  "provider": "gemini"        (solo per ai.apiKey)
}
```

plugin.init

```
payload: {  
  "version": "1.0.0",  
  "capabilities": ["ai", "daw", "plugin_control", "osc"]  
}
```

Descrizione: Handshake iniziale al caricamento UI

← Plugin verso UI (Server → Client)

ai.stream

```
payload: {  
  "chunk": "stringa di testo",  
  "isDone": false  
}
```

Descrizione: Chunk streaming risposta AI (un frame per parola/gruppo)

ai.response

```
payload: {  
  "status": "success|error|thinking",  
  "content": "testo completo",  
  "provider": "gemini",  
  "model": "gemini-1.5-pro",  
  "durationMs": 1240  
}
```

daw.transport

```
payload: {  
  "isPlaying": true,  
  "isRecording": false,  
  "bpm": 128.0,  
  "positionSeconds": 154.0,  
  "timeSignature": { "numerator": 4, "denominator": 4 }  
}
```

Frequenza: ogni comando DAW + ogni secondo durante play

daw.meter

```
payload: {  
  "trackId": -1,          (-1 = master)  
  "leftDb": -14.2,
```

```
"rightDb": -14.8,  
"peakLeftDb": -0.1,  
"peakRightDb": -0.3  
}
```

Frequenza: ~30fps durante playback, ~2fps in idle

plugin.state ← NUOVO (v1.1)

```
payload: {  
  "trackId": 1,  
  "pluginSlot": 0,  
  "pluginName": "FabFilter Pro-Q3",  
  "parameters": [  
    { "name": "Band1_Gain", "value": -4.5, "unit": "dB" },  
    { "name": "Band1_Freq", "value": 220.0, "unit": "Hz" }  
  ]  
}
```

agent.memory ← NUOVO (v1.1)

```
payload: {  
  "event": "observation|preference|action",  
  "note": "utente accetta tagli EQ narrow"  
}
```

Descrizione: Notifica UI che la memoria è stata aggiornata

plugin.error

```
payload: {  
  "code": "AI_TIMEOUT|PARSE_ERROR|PLUGIN_NOT_FOUND|...",  
  "message": "Descrizione leggibile",  
  "severity": "error|warning|info"  
}
```

4. Protocollo OSC — Indirizzi Standard

Per la comunicazione con Reaper e DAW OSC-compatibili:

```
/transport/play          → avvia playback  
/transport/stop          → ferma playback  
/transport/record        → toggle recording  
/transport/bpm [float]   → imposta BPM  
  
/track/[n]/volume [float] → volume traccia N (0.0-1.0)  
/track/[n]/pan [float]   → pan traccia N (-1.0 a +1.0)
```

```
/track/[n]/mute [int]      → mute 0/1
/track/[n]/solo [int]      → solo 0/1

/fx/[n]/[m]/param/[p] [f] → param P del plugin M su traccia N

/master/volume [float]     → volume master
/master/gain [float]       → gain master
```

5. Latenze e Performance Target

Tipo di messaggio	Target	Misurato (localhost)
WebSocket round-trip	< 5ms	2-3ms tipico
AI TTFB (primo chunk)	< 1000ms	600-800ms (Ollama)
AI TTFB (cloud)	< 600ms	300-500ms (Gemini)
Meter update (30fps)	33ms	33ms ✓
Transport broadcast	< 10ms	5ms ✓
Plugin param change	< 2ms	1ms ✓

6. Gestione Errori e Reconnection

STRATEGIA RECONNECTION

1. Connessione persa → stato: DISCONNECTED
2. UI mostra indicatore offline
3. Auto-retry ogni 2 secondi (max 10 tentativi)
4. Se non riesce → stato: ERROR, mostra messaggio
5. Al reconnect → invia plugin.init per handshake
6. Plugin risponde con stato corrente completo

→ Continua in: [Paper 06 — Roadmap di Implementazione](#)